AWS Database Blog

Migrate logins, database roles, users, and object-level permissions from Azure SQL Database to Amazon RDS for SQL Server

by InduTeja Aligeti, Jitendra Kumar, and Ramesh Babu Donti | on 24 MAY 2024 | in Intermediate (200), Multicloud, RDS for SQL Server, Security, Technical How-to | Permalink | Comments | Share

In this post, we demonstrate how to migrate SQL logins, database roles, users, and object-level permissions from Azure SQL Database to Amazon Relational Database Service (Amazon RDS) for SQL Server using T-SQL.

Within SQL Server, a SQL login acts as a security principal, allowing a user or application to connect to a SQL Server instance. It serves as a mechanism to manage access, permissions, user privileges, and monitor database activities. Migrating SQL logins is crucial during a database migration because they contain authentication information, access rights, and permissions essential for users and applications to connect and interact with databases. While native backup and restore operations are often sufficient for login migration, certain scenarios require additional steps to ensure a seamless migration, especially when migrating from Azure SQL Database and using data transfer methods SmartbulkCopy, AWS Database Migration Service (AWS DMS), or BCP to migrate data and schema separately

SQL Server security components overview

In SQL Server, a SQL login is a security principal that allows a user or application to connect to an instance. These SQL logins are associated with user accounts for the purpose of authentication. Authentication in SQL Server involves three key elements:

- Principals – These are entities capable of requesting resources within SQL Server. Principal access levels are contingent on their scope. We can categorize principles into two types based on their scope: server-level principals and database-level principals.
- Securables – These are server or database objects to which authentication can be granted, such as tables or endpoints. Securables are further categorized based on their scope: server scoped, database scoped, and schema scoped.
- Permissions – Permissions define control of securables to the principal. Principals can run specific operations on securables based on the permissions assigned. At the server level, permissions are managed through logins and server roles, whereas at the database level, permissions are regulated using database users and roles.

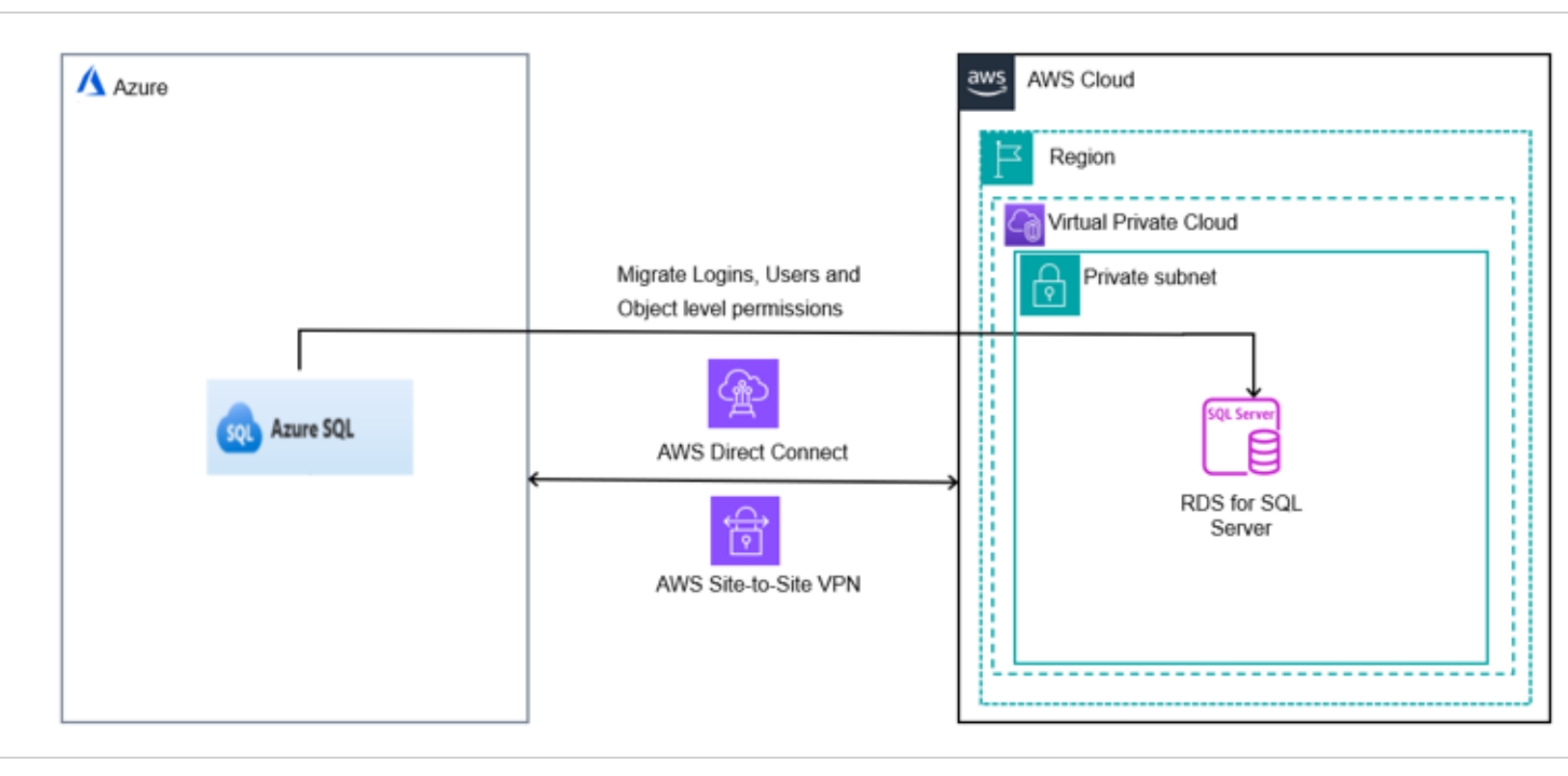
The following table illustrates the SQL Server security administration differences between Azure SQL Database and Amazon RDS for SQL Server.

Activity	Amazon RDS for SQL Server	Azure SQL Database
Where you manage server-level security	**Security** folder in SQL Server Management Studio's Object Explorer	`master` database and through the Azure portal
Windows Authentication	AWS Managed Microsoft AD	Azure Active Directory identities
Server-level security role for creating logins	`securityadmin` fixed server role	`loginmanager` database role in the master database
Commands for managing logins	`CREATE LOGIN, ALTER LOGIN, DROP LOGIN`	`CREATE LOGIN, ALTER LOGIN, DROP LOGIN` (there are some parameter limitations and you must be connected to the master database)
View that shows all logins	`sys.server_principals`	`sys.sql_logins` (you must be connected to the master database)
Server-level role for creating databases	`dbcreator` fixed database role	`dbmanager` database role in the master database

Solution overview

In scenarios where logins and user permissions are not migrated as part of the database migration process, such as when using SmartBulkCopy, AWS DMS, or BCP, we utilize the built-in generate script utility and T-SQL dynamic scripts to create the logins, roles, users, and object-level permissions for each database.

The following diagram illustrates the high-level architecture of migrating logins and database users from Azure SQL Database to Amazon RDS for SQL Server.



The following steps are involved to migrate the SQL logins and their dependent security components:

1. Migrate logins and validate the existence of logins and the users between Azure SQL Database to Amazon RDS for SQL Server.
2. Migrate server roles and server permissions and validate the permissions.
3. Migrate users and database roles and validate the users and the roles.
4. Migrate object-level permissions and validate the permissions.

Prerequisites

To get started, you need to have the following:

- An Azure SQL database as a source
- An RDS for SQL Server DB instance (Single-AZ or Multi-AZ) as a target
- Connectivity between Azure SQL Database to Amazon RDS for SQL Server
- A target RDS for SQL Server database with required user-defined objects (tables, views, functions, and stored procedures) and a user with the `db_owner` fixed database role to create objects in the SQL Server database
- An Azure login with minimum permissions to `master` to fetch login and user info like `loginmanager`, and more roles to make sure that the primary login name of Azure SQL Database is different from RDS for SQL Server

Migrate logins

You can access Azure SQL Database using a SQL Server authentication account, Microsoft Entra server principals, or a managed identity. Identify the Microsoft Entra server principals and managed identity by fetching the login information from `sys.logins` and migrate them manually. Ensure to create EXTERNAL_USERS as `db_owner` in Amazon RDS for SQL Server. Due to Amazon RDS for SQL Server's permission restrictions, you cannot grant sysadmin privileges to users at the target

To migrate SQL logins from Azure SQL Database to Amazon RDS for SQL Server, you can script out the create login scripts with a hashed password and SID. However, when using Azure SQL Database as the source, the generated SID and hashed password may cause conversion issues, and certain functions like `SUSER_SID()` are not supported.

In this section, we explain how to generate the create a login script from a source SQL Server instance and migrate it to the target RDS for SQL Server instance.

1. Connect to your source Azure SQL from SQL Server Manager Studio (SSMS) with the primary user or a user that has security admin fixed role permissions to script logins.
2. In the SSMS query editor, choose the **Results to Grid** (press **Ctrl+D**) option.
3. Run the following script from the new query window to generate the create logins script:

   ```
   --Script to generate create Logins
   Use master
   go
   SET NOCOUNT ON
   SELECT 'BEGIN CREATE LOGIN ' +QUOTENAME(sqllogin.name)+
   CASE
   WHEN sqllogin.type_desc = 'SQL_LOGIN' THEN ' WITH PASSWORD = ' +CONVERT(NVARCHAR(MAX),s
   + CASE WHEN sqllogin.is_expiration_checked = 1 THEN 'ON' ELSE 'OFF' END +', CHECK_POLIC
   ELSE ' NOT_SQL_LOGIN'
   END
   +' DEFAULT_DATABASE=[' +sqllogin.default_database_name+ '], DEFAULT_LANGUAGE=[' +sqllog
   AS [-- Amazon RDS Logins Creation Script --]
   FROM sys.sql_logins AS sqllogin
   ```

   The output of the query provides the logins creation script.
4. Copy the logins creation script and run it using the Amazon RDS primary user or a login with the "Alter any login" server-level securable that is part of a `processadmin` and `setupadmin` fixed server roles in the RDS for SQL Server instance.
5. Check for any errors and fix them accordingly.
6. Validate the number of logins created on the Azure SQL database and RDS for SQL Server database. To test the login if working properly, connect to RDS for SQL Server with any of the logins migrated with the password used to connect at the Azure SQL Database end.

After the successful migration of logins, you can continue with the server role membership migration.

You should be responsible and cautious while copying the output and pasting it at the target database because it contains sensitive data.

Migrate server roles and server permissions

In Amazon RDS for SQL Server, you don't have superuser or sysadmin privileges. The primary user is the highest privilege user. Users cannot be created in the master database. User creation happens at the database level. The primary user is specified during RDS instance creation with a user name and password. The primary user has the maximum privileges allowed within RDS, but not full sysadmin privileges. Only `setupadmin` and `processadmin` fixed server roles can be granted to logins by the primary user. Refer to the fixed server roles in the RDS for SQL Server instance for more details.

In Azure SQL Database, you can use the built-in server-level roles to manage permissions at the logical server level rather than granting permissions directly at that level. Refer to Azure SQL Database server roles for permission management for more details.

The key steps to migrate server-level roles and permissions from Azure SQL Database to an RDS for SQL Server instance are as follows:

1. For each login, identify the server-level roles they are a member of using the following script:

   ```
   SELECT m.name AS MemberName, r.name AS RoleName
   FROM sys.server_role_members rm
   JOIN sys.sql_logins m ON rm.member_principal_id = m.principal_id
   JOIN sys.server_principals r ON rm.role_principal_id = r.principal_id
   ```

2. After the logins are created in Amazon RDS for SQL Server, you can recreate the server roles for `setupadmin` and `processadmin` on the RDS instance, because other roles are not possible. Refer Azure SQL Database server roles for permission management and Microsoft Entra server principals for AD users. You can add the login as a member to the `processadmin` and `setupadmin` fixed server roles on RDS using the following statements:

   ```
   ALTER SERVER ROLE [processadmin] ADD MEMBER [<RDSLoginName>]
   GO
   ALTER SERVER ROLE [setupadmin] ADD MEMBER [<RDSLoginName>]
   GO
   ```

Refer to the permissions for the `processadmin` and `setupadmin` fixed server roles before manually adding logins to these roles on Amazon RDS.

Migrate users and database roles

To migrate your users and database roles, complete the following steps:

1. Connect to your source Azure SQL from SSMS with the primary user or user that has security admin fixed role permissions.
2. Choose (right-click) the database and choose **Generate Scripts**.
3. Choose **Next**.
4. Select **Select specific database objects**, then select **Users**.
5. Choose **Next**.
6. Select **Save as script file**.
7. Enter a location to save the file.
8. Choose **Next**.

   A script will be generated in the provided location.
9. Run the copied script on the target RDS for SQL Server database.
10. Validate the number of users, roles, and permissions on the Azure SQL database and match them with the RDS for SQL Server database by querying and joining the system tables `sys.database_principals, database_role_members, database_role_members,` and `sys.database_permissions`.

Migrate object-level permissions

Complete the following steps to migrate object-level permissions:

1. Connect to your source Azure SQL from SSMS with the primary user or user that has security admin fixed role permissions to script logins.
2. In the SSMS query editor, choose the **Results to Grid** (press **Ctrl+D**) option.
3. Run the following script from the new query window to generate the object-level permissions create script:

   ```
   USE <database_Name>
   -- Migrate the Server level permissions
   Select state_desc + ' ' + permission_name + ' ON [' + SCHEMA_NAME(SysObj.schema_id) + '
   +'] TO [' + USER_NAME(DBP.grantee_principal_id) + ']'+ '; ' [--Command to add object-le
   from sys.database_permissions DBP INNER JOIN sys.database_principals DBPS ON DBP.grante
   where DBPS.name not in ('public','dbo')
   ```

4. Run the created object-level permissions script on the target RDS for SQL Server database and validate.
5. Connect to the target Amazon RDS for SQL Server using SSMS and copy the output scripts generated in Step 4.
6. Validate the script output for any errors.

Clean up

To avoid future charges, remove all of the components you created while testing this use case by completing the following steps:

1. On the Amazon RDS console, choose **Databases** in the navigation pane.
2. Select the databases you set up and on the **Actions** menu, choose **Delete**.
3. Enter delete me to confirm deletion.

For more information about deleting an instance, refer to Deleting a DB instance.

Conclusion

In this post, we described how to migrate logins, database roles, users, and object-level permissions to Amazon RDS for SQL Server using T-SQL. If you have any questions or suggestions, leave a comment.

About the Authors

InduTeja Aligeti is a Senior Lead Database Consultant at AWS. She has over 18 years of experience working with Microsoft technologies with a specialization in SQL Server. She focuses on helping customers build highly available, cost-effective database solutions and migrate their large-scale SQL Server databases to AWS.

Jitendra Kumar is a Lead Database Migration Consultant with AWS Professional Services. He helps customers migrate and modernize workloads in the AWS Cloud, with a special focus on modern application architectures and development best practices.

Ramesh Babu Donti is a Lead Database Consultant with AWS Professional Services based out of Hyderabad, India. He focuses on helping customers build highly available, cost-effective database solutions and migrate their large-scale databases to AWS.

TAGS: Microsoft Azure SQL

Like Share

Comments

Log in to comment

Resources
Amazon Aurora
Amazon DocumentDB
Amazon DynamoDB
Amazon ElastiCache
Amazon Keyspaces (for Apache Cassandra)
Amazon MemoryDB
Amazon Neptune
Amazon Neptune Analytics
Amazon RDS
Amazon Timestream
AWS Database Migration Service

Learn
What Is AWS?
What Is Cloud Computing?
What Is Agentic AI?
Cloud Computing Concepts Hub
AWS Cloud Security
What's New
Blogs
Press Releases

Resources
Getting Started
Training
AWS Trust Center
AWS Solutions Library
Architecture Center
Product and Technical FAQs
Analyst Reports
AWS Partners

Developers
Builder Center
SDKs & Tools
.NET on AWS
Python on AWS
Java on AWS
PHP on AWS
JavaScript on AWS

Help
Contact Us
File a Support Ticket
AWS re:Post
Knowledge Center
AWS Support Overview
Get Expert Help
AWS Accessibility
Legal

Back to top

Amazon is an Equal Opportunity Employer: Minority / Women / Disability / Veteran / Gender Identity / Sexual Orientation / Age.
Privacy Site terms Cookie Preferences
© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.